

Lab 4: Consuming API in flutter

Rgs number: 223000471

Computer Engineering Year 3

Q₁ Which dependencies did you use and why?

My application use single external dependency: `http: ^1.2.1`

Why http and not dio?

- http is the official Dart team package
 - Lightweight
 - well maintained
 - sufficient for simple REST calls returning plain JSON.
 - no state management library (Provider, Riverpod, Bloc) was added deliberately.
 - no dio - It adds interceptors, correlation tokens, and advanced features that are not needed when consuming a single JSON endpoint.

Usage example:

```
final response = await http.get(Uri.parse(_baseUrl));  
if (response.statusCode == 200) {  
  final List<dynamic> jsonList = json.decode(response.body);  
  return jsonList.map((item) => Post.fromJson(item)).toList();  
} else {  
  throw Exception('Failed to load posts: ${response.statusCode}');  
}
```

Q₂ How can you handle API exception (Network errors, invalid data)?

Answer: Exception handling is applied to three distinct layers so that no error can crash the app unhandled:

• Layer 1: Service Layer

Every method checks the HTTP status code. Any non-2xx response throws a Dart Exception converting HTTP error into typed exceptions that propagate up.

• Layer 2: Future Builder

The `snapshot.hasError` branch of `FutureBuilder` catches exceptions thrown by `getPost()` and renders a friendly error UI with a Retry button instead of crashing the application.

• Layer 3: try/catch in Button handler

All onpressed callbacks that invoke the service are wrapped in `try/catch`. Failures are displayed as a red Snack Bar, and the finally block always resets the loading spinner.

Additional safeguards:

- `int.tryParse()` in the form validator prevents a crash if the user type non-numeric text in the user ID field.
- `context.mounted` is checked after every `await` to avoid calling `setState` or `ScaffoldMessenger` on a widget that has already been disposed.
- Network unavailability is handled automatically - the `Http` package throws a `SocketException` which propagates as an `Exception` to layer 4/3.

Q3 Explain the `FutureBuilder` widget you used and the UI that depends on it.

`FutureBuilder<T>` is a Flutter built-in widget that listens to `Future<T>` and automatically rebuilds its subtree each time the `Future`'s state changes. It exposes as `AsyncSnapshot<T>` to its builder callback, which contains both the connection state and the data (or errors).

Where it is used:

- It is used in `PostListScreen` to drive the initial data load. It watches `_postFuture` (a field set in `initState`) and renders one of three UI states:
 - Loading
 - Error
 - Success
- Refresh pattern;
The app also maintains a local - posts list. `FutureBuilder` handles loading and error states; once data is loaded, `_posts` is used as the source of truth. Mutations (create / update / delete) update `_posts` locally so changes appear instantly without a redundant network round-trip.